

1. INTRODUCCIÓ I CONTEXTUALITZACIÓ

El projecte “Creació i Desplegament de Solucions Empresarials amb IBM BAMOE” està definit com un Treball de Final de Grau de la titulació del grau en Enginyeria Informàtica de la Facultat d’Informàtica de Barcelona (FIB), Universitat Politècnica de Catalunya (UPC). Aquest treball és de modalitat B i ha estat realitzat amb col·laboració de l’empresa Capgemini a través d’un conveni de cooperació educativa.

1.1. Context

El principal objectiu de qualsevol empresa, sigui de l’àmbit que sigui, és evolucionar contínuament per mantenir-se competitiva en un mercat que es troba en constant canvi. Per aconseguir-ho, l’optimització de processos ha esdevingut un factor clau, ja que permet reduir costos, augmentar la productivitat i redistribuir millor els recursos de l’empresa. Per aquest motiu, cada cop més empreses adopten eines tecnològiques que els ajuden a automatitzar tasques i a prendre decisions més àgils i precises. [1]

En aquest context, neix Business Automation Manager Open Edition (BAMOE), una plataforma de codi obert i basada en el núvol desenvolupada per IBM que permet a les empreses automatitzar processos, millorar la presa de decisions i adaptar-se ràpidament als canvis del mercat. La seva flexibilitat i escalabilitat la converteix en una solució idònia per a diferents sectors, impulsant la transformació digital i l’optimització de la gestió empresarial.

Aquest treball se centrarà en l’ús d’aquesta tecnologia per donar resposta a un dels grans reptes del sector financer: l’aprovació de préstecs hipotecaris des d’una perspectiva operativa. Tradicionalment, aquest procés es duu a terme de manera manual i burocràtica, fet que implica que múltiples agents, com gestors bancaris i analistes de crèdit, intervinguin en el procés i, en conseqüència, s’incrementi el temps de la resposta que espera el client.

El sector hipotecari a Espanya és un component fonamental de l’economia. De fet, el nombre d’hipoteques sobre habitatges va augmentar un 11,2% el 2024, assolint 423.761 operacions. [2] Aquest creixement va convertir el 2024 en el segon millor any en termes de concessions hipotecàries des de la crisi financera.

Segons dades de l’Institut Nacional d’Estadística (INE), l’augment en la demanda de préstecs es deu principalment a la caiguda del tipus d’interès de l’Euríbor, que va passar del 3,6% al gener al 2,4% el desembre de 2024, reduint així el cost del finançament per als compradors. A més, l’import mitjà de les hipoteques constituïdes va situar-se en 152.377 €, un fet que indica una

millora en l'accessibilitat al crèdit. A banda d'això, les execucions hipotecàries sobre habitatges es van reduir un 4,3% respecte al 2023, reflectint una major estabilitat econòmica i confiança en el mercat immobiliari. [3]

1.2. Conceptes previs

En aquest apartat es farà una breu explicació de diferents conceptes rellevants en el TFG i, per tant, cal tenir presents al llarg de la lectura de la memòria per entendre-la correctament.

BAMOE (*Business Automation Manager Open Edition*): Desenvolupat per IBM, BAMOE es tracta d'un *software* d'automatització empresarial per la gestió de fluxos de treball i decisions basades en regles de negoci. [4]

DMN (*Decision Model and Notation*): Llenguatge de modelatge i notació per l'especificació precisa de decisions i regles de negoci. [5]

BPMN (*Business Process Model and Notation*): Estàndard per a la modelització de processos empresarials mitjançant diagrames de flux de treball. Consisteix en una unitat lògica que agrupa components que permeten definir un flux de treball, cosa que permet als usuaris crear lògica dins d'un procés de negoci i realitzar la integració amb altres aplicacions i fonts de dades. [6]

PMML (*Predictive Model Markup Language*): Llenguatge de marcació, basat en aprenentatge autònom, utilitzat per definir i compartir models de mineria de dades entre aplicacions, independentment del fabricant. [7]

Préstec hipotecari: Contracte a través del qual una entitat atorga una quantitat de diners determinada a una empresa o particular per a l'adquisició d'un immoble a canvi d'uns interessos determinats i durant un termini establert. [8]

Avaluació creditícia: Procés d'anàlisi de la solvència d'un sol·licitant de préstec mitjançant una revisió de la seva informació financera i historial creditici, és a dir, de les seves fonts d'ingressos, deutes pendents i activitats creditícies recents.

Historial creditici: Informe que agrupa tota la informació financera d'un client, de manera que inclou un expedient detallat dels deutes que té en l'actualitat i també possibles impagaments que pot haver tingut en el passat i l'actualitat. [9]

1.3. Descripció del problema

L'aprovació d'un préstec hipotecari és un procés fonamental en el sector financer per a moltes persones que volen accedir a un habitatge, però sovint depèn de procediments manuals que obstaculitzen el seu bon funcionament. El procés d'aprovar una petició d'aquest tipus no és instantani i pot allargar-se diverses setmanes a causa de tots els requisits i processos manuals que s'han de complir per obtenir-ho. Segons Banc Sabadell, el termini per a la concessió d'una hipoteca pot oscil·lar entre 15 i 40 dies, depenent de la rapidesa en la tramitació de la documentació, la verificació d'aquesta, l'anàlisi de risc i de les condicions de cada entitat financera. [10]

Aquest retard no només afecta els clients, generant incertesa i possibles abandonaments del procés, sinó que també impacta en l'eficiència operativa de les entitats financeres que han de gestionar un gran nombre de sol·licituds. A més, la manca d'automatització en l'anàlisi creditícia i la presa de decisions pot donar lloc a errors humans, aprovant préstecs a clients no aptes o rebutjant sol·licituds viables.

D'aquesta manera, els llargs temps de processament, la manca d'informació en temps real i la falta d'estandardització en la presa de decisions generen ineficiències i desconfiança en els clients i, a la vegada, eleven el cost operatiu de la gestió financera.

1.4. Actors implicats

Per donar solució al problema comentat en l'apartat anterior, cal considerar les persones que estan involucrades directament o indirecta en el procés per automatitzar les peticions d'un préstec hipotecari. Aquests *stakeholders* tenen un paper fonamental en el desenvolupament del sistema, ja que les seves necessitats i requisits determinen les funcionalitats i l'abast de la solució. Així doncs, els principals actors implicats en el projecte són els següents.

Sol·licitants de préstecs

Els sol·licitants dels préstecs són les persones que busquen finançament per adquirir un immoble. Aquests usuaris presenten una sol·licitud i esperen una resposta ràpida i transparent.

Entitats financeres

Les entitats financeres són els bancs i organitzacions que ofereixen crèdits hipotecaris i necessiten fer ús d'un sistema eficient per aprovar els préstecs. Amb l'automatització, podran optimitzar el procés d'aprovació, reduint costos operatius i minimitzant errors en la presa de decisions per ser més eficients i competitius.

Analistes de crèdit

Els analistes de crèdit són els responsables d'examinar la capacitat financera dels sol·licitants i determinar si compleixen els requisits per obtenir un préstec hipotecari. Per aquests *stakeholders*, aquesta eina els servirà per estandarditzar les avaluacions i reduir la subjectivitat, millorant la precisió en la presa de decisions.

Director del Treball de Fi de Grau

Els directors d'aquest Treball de Fi de Grau són Agustín Botana i Jesús Pérez, treballadors de l'empresa Capgemini, i són els encarregats de supervisar el desenvolupament del projecte perquè es compleixin els objectius plantejats.

Ponent del Treball de Fi de Grau

El ponent d'aquest treball és en Carles Farré, encarregat de fer un assessorament acadèmic del projecte i correcció final del treball.

Desenvolupadora del TFG

L'autora d'aquest projecte també és una *stakeholder*, ja que s'encarrega de desenvolupar de manera òptima el projecte, participant en el desenvolupament, validació i documentació d'aquest. A més a més, el projecte suposa una oportunitat de creixement tant acadèmica com professional, donat que s'adquirirà experiència en la tecnologia innovadora, BAMOE.

8. ESPECIFICACIÓ DE REQUISITS

L'apartat d'especificació de requisits defineix les històries d'usuari i el model de dades emprats per desenvolupar aquest projecte.

8.1. Històries d'usuari

El projecte s'especifica a partir de les històries d'usuari que defineixen els requisits funcionals del treball. Aquestes històries tenen en compte el rol que hi ha al sistema: l'empleat del banc que, sent l'únic usuari de l'aplicació, pot actuar com a gestor o analista.

A continuació s'especifiquen les històries d'usuari.

US1. Registre d'una nova sol·licitud

Com a empleat del banc,

vull poder registrar una nova sol·licitud de préstec amb dades bàsiques d'aquesta i l'identificador del client,

per tal d'iniciar el procés d'avaluació de la petició.

Criteris d'acceptació:

- L'usuari pot accedir a un formulari per introduir les dades bàsiques i rellevants de la sol·licitud (ID del client, import del préstec, termini del préstec, taxa d'interès i tipus de préstec).
- En prémer "Guardar", el sistema inicia l'execució del flux on, si l'identificador del client és vàlid, el sistema accedeix a un microservei per recuperar les seves dades. En el cas que el client no existeixi, es mostra un missatge d'error i no s'inicia el flux.
- En el cas en què no hi ha errors amb el client, es crea un nou ítem de la sol·licitud a la base de dades i es mostra una confirmació.
- Una vegada enregistrada, l'aplicació redirigeix a la pantalla de detalls de la sol·licitud, mostrant les dades d'aquesta.

US2. Consultar sol·licitud

Com a empleat del banc,

vull poder consultar una sol·licitud enregistrada,

per tal de veure el valor dels seus paràmetres i poder fer un seguiment d'aquesta.

Criteris d'acceptació:

- L'usuari pot cercar sol·licituds a partir de l'identificador de la sol·licitud.
- El sistema mostra els detalls complets de la sol·licitud.
- Si la sol·licitud no existeix, es mostra un missatge d'error.

US3. Visualitzar el llistat de sol·licituds

Com a empleat del banc,

vull poder consultar el llistat de sol·licituds enregistrades i aplicar-hi un filtratge segons la decisió i accions d'aquestes,

per tal de veure totes les sol·licituds i els seus estats respectius.

Criteris d'acceptació:

- Es poden aplicar filtres per decisió (aprovada, rebutjada, pendent), per acció (validació documentació, pendent documentació, en avaluació, aprovada, rebutjada, en avaluació PMML, en revisió manual, notificant, completada) i es pot cercar per identificador de sol·licitud.
- El sistema mostra el llistat filtrat de sol·licituds.
- Si no hi ha sol·licituds que compleixen els filtres, es mostra un missatge informatiu.

US4. Validació automàtica de la documentació

Com a empleat del banc,

vull que el sistema validi automàticament si la documentació està completa i compleix uns requisits determinats,

per tal d'evitar iniciar l'avaluació amb dades incorrectes.

Criteris d'acceptació:

- El sistema comprova que tots els paràmetres requerits estan presents i són correctes.
- Si falta algun valor o algun és incorrecte, l'estat de la sol·licitud canvia a "Documentació pendent" i es notifica a l'usuari que ha de revisar manualment aquesta documentació.
- Si tot és correcte, s'avança automàticament a la fase d'avaluació.

US5. Editar una sol·licitud

Com a empleat del banc,

vull poder modificar les dades incompletes o incorrectes d'una sol·licitud,

per tal que passin la validació de la documentació.

Criteris d'acceptació:

- Només es poden editar les sol·licituds en estat “Pendiente documentación” o “Revisión manual”.
- L'usuari només pot editar uns camps en específic depenent de la tasca manual que hagi de realitzar.
- El sistema valida que les dades modificades siguin vàlides abans de guardar-les.
- Els canvis s'emmagatzemen i es mostra una confirmació d'actualització.

US6. Avaluació automàtica del risc creditici

Com a empleat del banc,

vull que el sistema faci una avaluació automàtica del risc mitjançant regles de decisió DMN i, si cal, un model PMML,

per tal de determinar el risc d'acceptar aquesta sol·licitud.

Criteris d'acceptació:

- El sistema aplica un DMN per fer una primera classificació del risc (baix, mitjà, alt).
- Si el risc és alt, el sistema determina que la decisió ha estat rebutjada.
- Si el risc és baix, el sistema determina que la decisió ha estat aprovada.
- Si el risc és mitjà, s'executa el model PMML i es fa una revisió manual.

US7. Revisió manual del risc

Com a empleat del banc,

vull revisar manualment les sol·licituds que han rebut un risc mitjà per part de la taula de decisions,

per tal d'assegurar que s'obté un bon resultat de la decisió.

Criteris d'acceptació:

- El sistema permet que l'analista entri i modifiqui la decisió final de la sol·licitud.
- L'usuari pot veure les dades d'entrada del model i de la sortida del PMML abans de decidir.

US8. Consultar estadístiques de sol·licituds

Com a empleat del banc,

vull poder consultar el nombre de sol·licituds aprovades, rebutjades i pendents de tramitar,

per tal de veure la quantitat de cada una.

Criteris d'acceptació:

- El sistema mostra el nombre de sol·licituds aprovades, rebutjades i pendents.

US9. Visualització del resultat de l'avaluació

Com a empleat del banc,

vull visualitzar el resultat de l'avaluació i els motius de la decisió,

per tal de justificar l'aprovació o el rebuig.

Criteris d'acceptació:

- El sistema mostra el resultat de la decisió (aprovat o rebutjat).
- Es mostra un resum amb els motius d'aquesta decisió, incloent-hi l'estat final, la decisió presa i les regles que s'han aplicat i la sortida del model PMML, si escau.

US10. Llistar notificacions

Com a empleat del banc,

vull veure les notificacions generades,

per tal de saber quin és l'estat actual de la sol·licitud o saber si he de fer una tasca manual.

Criteris d'acceptació:

- El sistema mostra les notificacions, que es poden filtrar per veure-les totes o només les no llegides.
- Les notificacions es generen després d'executar el model predictiu, per indicar a l'analista de crèdit que ha de revisar manualment la sol·licitud, quan s'ha de validar la documentació d'una sol·licitud perquè és incorrecta, i per informar que la sol·licitud ha estat aprovada o rebutjada.

US11. Marcar notificació com a llegida

Com a empleat del banc,

vull poder marcar com a llegida una notificació,

per tal de deixar només les que no he llegit i tenir millor control d'aquestes.

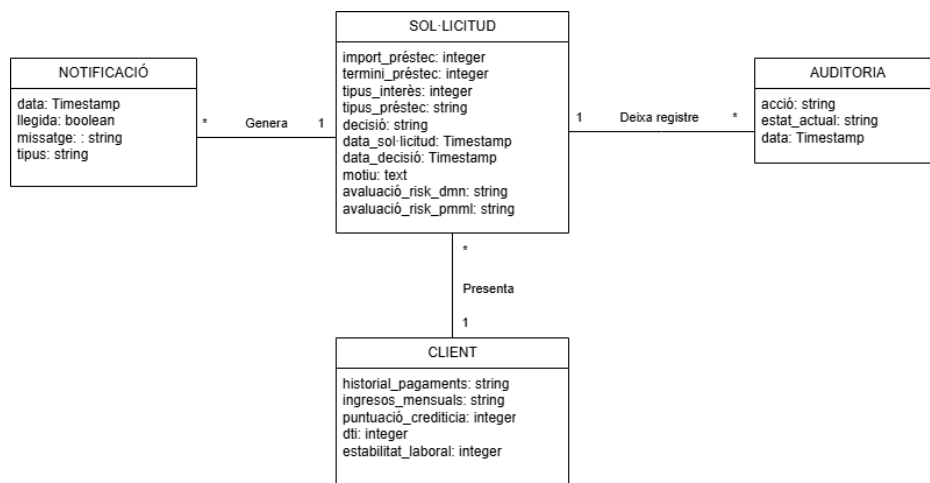
Criteris d'acceptació:

- En prémer el botó “Marcar leída” de la notificació o accedir als detalls de la sol·licitud a través de la notificació, aquesta es marca com a llegida.

8.2. Model conceptual

El model conceptual representa les entitats que intervenen en el sistema de gestió de sol·licituds de préstecs hipotecaris i les seves relacions. Les entitats identificades al sistema són:

- Sol·licitud: representa una petició feta per un client. Inclou atributs com l'import sol·licitat, el tipus i el termini del préstec i les dades relacionades amb l'avaluació del risc.
- Notificació: inclou els missatges interns generats pel sistema en el moment que l'empleat del banc ha de completar la informació del client abans de començar l'avaluació, ha de fer una revisió manual de la petició o vol informar que la petició tractada ha estat aprovada o rebutjada.
- Client: representa la persona sol·licitant del préstec. Inclou la informació sobre el seu perfil financer, com els ingressos mensuals o la seva estabilitat laboral, entre d'altres.
- Auditoria: emmagatzema les accions realitzades sobre cada sol·licitud per garantir traçabilitat.



*Figura 3. Model conceptual de dades
Font: elaboració pròpia*

Restriccions textuais:

1. Una Sol·licitud ha d'estar sempre vinculada a un client existent.
2. Només es poden generar notificacions per a sol·licituds que estiguin en curs (no completades).
3. El valor de puntuació_crediticia ha d'estar entre 0 i 850.
4. El valor de dti ha d'estar entre 0 i 100.
5. El valor de historial_pagaments ha de ser: Puntuals, Tardans o Freqüents.
6. El valor de ingressos_mensuals ha de ser: Suficients, Insuficients o Justos.
7. La data_decisió ha de ser major o igual a data_sol·licitud.
8. El valor de decisió ha de ser: Aprovada, Rebutjada o Pendent.
9. El valor de tipus ha de ser: Aprovada, Rebutjada o Pendent.
10. El valor d'acció ha de ser: "Validación documentación", "Pendiente documentación", "En evaluación", "Aprobada", "Rechazada", "En evaluación PMML", "Revisión manual", "Notificando", "Completada".
11. El valor d'estat_actual ha de ser: Aprovada, Rebutjada o Pendent.

8.3. Requisits no funcionals

Els requisits no funcionals del sistema són aquells que determinen la qualitat del *software* i tenen en compte aspectes d'escalabilitat, seguretat i rendiment, entre d'altres. En aquest apartat es detallaran els criteris d'acceptació perquè es consideri que s'han complert els requisits no funcionals esmentats a l'apartat 3.2. *Requisits*.

ID	NFR1. Rendiment i eficiència operativa
Descripció	El sistema ha de permetre el processament simultani de múltiples sol·licituds sense disminuir la capacitat del rendiment.
Justificació	És possible que el volum de peticions augmenti en moments concrets, com pot ser en hores puntes, però, tot i això, cal garantir continuïtat operativa i bona experiència d'usuari per mantenir una eficiència alta en tot moment.
Criteris d'acceptació	- El sistema pot gestionar almenys 50 sol·licituds concurrents amb un temps de resposta inferior a 2 segons per operació. - El sistema manté un rendiment estable en entorns locals, amb possibilitat de tenir el mateix comportament en entorns de producció escalables.

Taula 14. NFR1: Rendiment i eficiència operativa
Font: elaboració pròpia

ID	NFR2. Escalabilitat
Descripció	La infraestructura ha de permetre un creixement dinàmic d'usuaris i sol·licituds segons les necessitats del moment.
Justificació	La demanda pot anar variant i és important que el sistema pugui evolucionar a llarg termini.
Criteris d'acceptació	- S'han de poder afegir instàncies o recursos addicionals sense modificar el codi base. - El back-end i el microservei poden escalar de manera independent segons la càrrega de treball.

Taula 15. NFR2: Escalabilitat
Font: elaboració pròpia

ID	NFR3. Usabilitat i accessibilitat
Descripció	La interfície ha de ser intuïtiva i fàcil d'utilitzar per als usuaris amb perfil no tècnic.
Justificació	Els empleats del banc han de poder registrar, revisar i consultar sol·licituds, així com visualitzar l'estat del préstec o intervenir en l'aprovació d'aquest quan sigui necessari, de forma fàcil sense molta formació prèvia.
Criteris d'acceptació	- Hi ha validacions clares i missatges d'error útils davant d'entrades incorrectes i hi ha informació clara dels camps que s'han d'omplir. - El temps mitjà per registrar una sol·licitud no supera els 2 minuts per a un usuari no expert.

Taula 16. NFR3: Usabilitat i accessibilitat
Font: elaboració pròpia

ID	NFR4. Compatibilitat
Descripció	La interfície de l'aplicació ha de ser compatible amb els navegadors corporatius i sistemes existents del banc.
Justificació	Tot el personal de l'empresa que ha de tenir accés a la plataforma, ho ha de poder fer independentment del dispositiu o sistema corporatiu que tingui.
Criteris d'acceptació	- L'aplicació es renderitza, és a dir, es mostra correctament al navegador i funciona amb les últimes 2 versions de Chrome, Edge i Firefox en sistemes Windows. - L'aplicació pot interaccionar amb sistemes interns, com bases de dades i microserveis, sense errors ni adaptacions manuals forçades.

Taula 17. NFR4: Compatibilitat
Font: elaboració pròpia

ID	NFR5. Disponibilitat
Descripció	El sistema ha d'assegurar alta disponibilitat durant l'horari laboral, minimitzant el temps d'inactivitat.
Justificació	Tenir un accés continu a la plataforma és essencial per assegurar que no hi ha interrupcions en la gestió de sol·licituds.
Criteris d'acceptació	- La disponibilitat ha de ser igual o superior al 99% en horari laboral. - El sistema ha d'incloure mecanismes d'alerta per detectar caigudes i recuperar el servei de forma ràpida, tenint plans de contingència definits.

Taula 18. NFR5: Disponibilitat
Font: elaboració pròpia

ID	NFR6. Temps de resposta
Descripció	El sistema ha de processar les sol·licituds, garantint respostes ràpides en l'anàlisi dels riscos.
Justificació	Els usuaris esperen utilitzar un sistema que no dificulti el desenvolupament de la seva feina.
Criteris d'acceptació	- El temps mitjà de resposta dels <i>endpoints</i> REST és inferior a 2 segons en condicions normals, com consultar o modificar una sol·licitud o llistar-les. - El temps de resposta per una anàlisi de risc amb el model DMN o PMML ha de ser menor de 3 segons.

Taula 19. NFR6: Temps de resposta
Font: elaboració pròpia

ID	NFR7. Compliment normatiu
Descripció	El sistema ha d'estar alienat amb les regulacions financeres com GDPR (Protecció de Dades) i PCI DSS (Seguretat de Transaccions Financeres), assegurant el compliment.
Justificació	Tractar amb dades de clients implica que cal assegurar la privacitat i seguretat d'aquestes per evitar sancions legals i mantenir la confiança del client.
Criteris d'acceptació	- El sistema ha de disposar de mecanismes d'auditoria que registrin qui accedeix a la informació i quan. - El sistema ha de disposar d'una capa de seguretat d'accés. - Es revisarà que totes les funcionalitats compleixin amb normativa GDPR i PCI DSS abans de posar el sistema en producció.

Taula 20. NFR7. Compliment normatiu
Font: elaboració pròpia

9. ARQUITECTURA DEL SISTEMA

9.1. Arquitectura física i lògica

Per gestionar i automatitzar el procés de sol·licituds de préstecs hipotecaris, el sistema es basa en una arquitectura modular que es pot analitzar des de dues perspectives. Per un costat, l'arquitectura física, que defineix on i com s'executen els components del sistema. Per l'altre costat, l'arquitectura lògica, que estableix com està organitzat funcionalment el sistema i com interactuen els seus components.

A nivell físic, el sistema s'ha desenvolupat i executat en un entorn local, utilitzant l'ordinador personal com a entorn d'execució. Inicialment, es va considerar utilitzar un Docker² per gestionar els contenidors i desplegar els components del projecte, però avaluant les característiques de Kogito, s'ha vist que utilitzant Kogito³ era suficient per executar el projecte en local sense necessitat de tenir un aïllament addicional, de manera que no s'ha emprat.

Els principals components de l'arquitectura física són els següents:

- Back-end principal: desenvolupat amb Quarkus, gestionat amb Maven i integrat amb BAMOE i Kogito.
- Front-end: desenvolupat amb Angular i consumeix els serveis implementats al back-end a través d'una API REST.
- Bases de dades relacionals (PostgreSQL): persisteix informació de les sol·licituds, notificacions i accions d'auditoria.
- Microservei de client: desenvolupat amb Spring Boot amb la finalitat de proporcionar la informació necessària d'un client a partir del seu identificador.

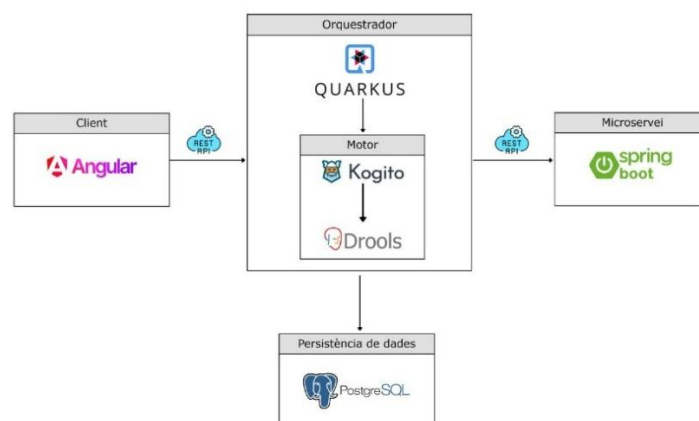


Figura 4. Esquema de l'arquitectura física del sistema
Font: elaboració pròpia

² Docker és un projecte de codi obert que automatitza el desplegament d'aplicacions dins de contenidors de programari per proporcionar una capa addicional d'abstracció. [26]

³ Explicat a l'apartat 9.2. *Tecnologies del back-end*.

A nivell lògic, l'arquitectura es basa en la separació de responsabilitats entre els diferents components del sistema:

- Front-end (Angular): interfície a partir de la qual l'usuari registra, consulta i gestiona les sol·licituds.
- Back-end (Quarkus i Kogito): orquestra el procés de negoci, la gestió de les sol·licituds i la persistència.
- Kogito: proporciona un entorn d'execució per desplegar processos BPMN i regles DMN.
- Microservei (Spring Boot): accedeix a la informació dels clients, retornant-la al back-end principal per avaluar el risc de la petició.
- Base de dades (PostgreSQL): emmagatzema les dades del sistema.

La comunicació entre els components es realitza mitjançant peticions HTTP/REST per garantir una arquitectura desacoblada i extensible.

9.2. Tecnologies del back-end

El back-end del sistema gestiona la lògica relacionada amb el registre i l'avaluació de sol·licituds de préstecs, així com l'orquestració del procés definit mitjançant BPMN.

Pel que fa als requisits tècnics necessaris per al desenvolupament i l'execució correcte del back-end, es requereix:

- Versió 17 de Java: aquesta versió de JDK assegura la compatibilitat amb Kogito i Quarkus.
- Versió 3.9.9. de Maven: per gestionar les dependències, compilar i empaquetar el projecte. És important afegir la variable d'entorn `MAVEN_HOME` perquè funcioni correctament en el sistema operatiu.
- Configuració del PATH per reconèixer les comandes de Maven i Java.
- IDEs: IntelliJ IDEA i VSCode per descarregar *plugins* per utilitzar Quarkus i modificar arxius BPMN i DMN.

A més, s'ha dissenyat una arquitectura basada en tecnologies *cloud-native*⁴ amb un component de modelatge de processos i d'automatització de decisions explicades seguidament.

⁴ El núvol natiu és un patró d'arquitectura del *software* que utilitza tecnologies com Kubernetes, contenidors i microserveis per a desenvolupar aplicacions al núvol que es puguin executar a dins d'un entorn de *cloud computing*. [27] El *cloud computing* implica entregues de dades, aplicacions i poder de processament als clients a través d'internet, eliminant la necessitat d'infraestructures físiques en l'extrem del client.

A continuació s'expliquen de forma detallada quines han estat les tecnologies i els components emprats en el projecte.

BAMOE

IBM BAMOE (Business Automation Manager Open Editions) és una plataforma *cloud-native* de gestió de processos i decisions empresarials, consolidada a partir d'una evolució de les plataformes de Red Hat, conegudes com a *Red Hat Process Automation Manager* i *Decision Manager*. Així doncs, es tracta d'una solució integral, compatible amb entorns *on-premise*⁵ i *cloud-native*, que es pot desplegar en múltiples plataformes gràcies al seu disseny basat en contenidors. [29]

BAMOE és una plataforma que permet als usuaris de negoci i tècnics definir, gestionar, validar i desplegar processos i regles de forma modular i visual. D'aquesta manera, utilitza un repositori centralitzat que manté la consistència i traçabilitat d'aquestes decisions i processos, combinant de forma nativa tres pilars fonamentals. D'una banda, la gestió de processos de negoci (BPMN), cosa que permet modelar gràficament el flux complet d'un procés, definir les seves tasques, condicions, bifurcacions i participants implicats. D'altra banda, a través de les taules de decisions DMN i el motor de regles Drools, es poden definir criteris de decisió clars que es poden canviar sense necessitat de modificar el codi, i això permet una resposta àgil a canvis en la política d'una empresa o regulacions externes. Finalment, per a processos on no se segueix un flux totalment predefinit, la gestió de casos permet establir escenaris flexibles per poder iniciar i completar activitats en funció de l'evolució del cas o de les intervencions humanes específiques. [30]

S'ha escollit aquesta tecnologia perquè permet una integració nativa amb Kogito i amb això es poden executar processos i regles de forma eficaç dins del back-end. A més a més, és compatible amb estàndards oberts com BPMN i DMN, té una arquitectura modular i escalable que s'adapta a microserveis i entorns distribuïts i garanteix la interoperabilitat i l'evitació del bloqueig amb un únic proveïdor (*vendor lock-in*⁶).

BAMOE conté diferents components, però els que es destaquen en aquest treball són: Drools i Kogito. De forma resumida, BAMOE proporciona un entorn d'execució per als processos del back-end. A través de Kogito, aquests processos s'executen de manera modular i és Drools qui aplica les regles de negoci definides.

⁵ Model de llicència i ús de *software* on les empreses utilitzen el *software* en el seu propi hardware (aplicacions en local). [28]

⁶ El bloqueig del proveïdor fa que un client depengui d'un venedor per als productes i no pugui utilitzar un altre proveïdor sense costos de canvi substancials. [31]

DROOLS

Drools es tracta d'un *Business Rules Management System* (BRMS) amb un motor de regles a partir del qual es gestionen regles, és a dir, executa una lògica de negoci definida per l'usuari en forma de regles declaratives. Té una arquitectura que es basa en el paradigma *if-then*, on s'estableixen condicions i conseqüències.

Drools utilitza un motor optimitzat per executar moltes regles a la vegada. Així doncs, es carreguen dades en la memòria de treball, es comparen amb totes les regles, i totes aquelles regles que coincideixen es col·loquen en una agenda d'execució que executa les regles, seguint una política, que per defecte executa primer la regla més recent. Amb aquest enfocament, s'aconsegueix detectar múltiples coincidències al mateix temps, prendre decisions complexes i evitar repeticions innecessàries. [32]

En aquest projecte, Drools executa les regles de negoci definides en format DMN, de manera que executa les taules internament i aplica el mateix principi: compara els *inputs* amb les condicions i aplica les sortides corresponents.

KOGITO

Kogito és una tecnologia d'automatització de negoci dissenyada per entorns *cloud-native* que permet crear aplicacions que es poden executar en entorns híbrids o al núvol. Es tracta d'una evolució del motor Drools i forma la base tècnica de BAMOE. [33]

Està basada en la modularitat i escalabilitat, ja que es poden desplegar serveis independents per a cada procés o regla. A més a més, cada procés es converteix en un servei REST personalitzat segons les dades del domini del negoci i està pensat per arrencar ràpidament, és a dir, que amb un baix ús de recursos, els microserveis i contenidors arranquen de forma ràpida.

En el projecte, Kogito s'ha integrat per executar processos BPMN, DMN i PMML explicats en els apartats posteriors.

QUARKUS

Quarkus és un *framework* Java de codi obert que serveix per a desenvolupar aplicacions de forma nativa al núvol. Així doncs, és un marc Java natiu de Kubernetes⁷ que té l'objectiu de facilitar

⁷ Kubernetes és una plataforma de codi obert per administrar càrregues de treball i serveis. [34]

molt el treball als desenvolupadors, gràcies a la seva capacitat de reduir dràsticament el temps d'arrencada i el consum de memòria. [35]

Aquest *framework* actua com a base del sistema, ja que és el contenidor principal on s'integren l'API REST, s'executen els processos mitjançant Kogito i es gestionen les sol·licituds, notificacions i auditories a través dels controladors, serveis i repositoris implementats.

MAVEN

Maven és una eina que té com a objectiu principal permetre al desenvolupador entendre quin és l'estat complet d'un projecte de desenvolupament en el menor temps possible. Dit amb altres paraules, és una eina de construcció i gestió utilitzada en entorns Java per automatitzar el cicle de vida del projecte, tenint en compte la descàrrega de dependències, la compilació, els tests i l'empaquetament final de l'aplicació. [36]

A través del fitxer *pom.xml*, se centralitzen totes les dependències, ja que permet incloure biblioteques com Quarkus, Kogito, Drools, RESTeasy i PostgreSQL. A més a més, empaqueta arxius JAR⁸, i això permet executar en local el back-end i integra mòduls externs com *Kogito workflows* i *Drools rules*.

BPMN

BPMN és un llenguatge estàndard de modelització de processos de negoci que s'utilitza de forma gràfica per definir els diferents passos d'un procés i facilitar la comprensió per part dels *stakeholders* tècnics i no tècnics. Utilitza una notificació gràfica clara que comprèn tasques, *gateways*, esdeveniments i fluxos seqüencials.

Els BPMN s'han d'executar en uns *plugins* compatibles i és gràcies a *Kogito Developer Tools*, que es troba a *VSCode*, que es poden crear i editar arxius amb extensió *.bpmn* correctament. Tot i això, cal assegurar que l'extensió de Kogito per a BPMN està inclosa a l'arxiu *pom.xml*.

Els processos BPMN poden tenir diferents versions per mantenir un control de canvis, però pel flux implementat al treball, només s'ha fet una única versió.

⁸ Aquest tipus d'arxiu comprimit conté totes les classes compilades, recursos i dependències d'una aplicació Java.

DMN

DMN és una notació estàndard desenvolupada per *Object Management Group* (OMG) que serveix per definir regles de negoci i models de decisió a través d'una lògica declarativa. Des del punt de vista arquitectònic, el model DMN s'integra com una tasca de decisió dins del procés BPMN i es gestiona i executa a partir de Kogito dins del back-end Quarkus. Cal remarcar que les regles definides al model estan totalment desacoblades del codi Java i es poden modificar sense necessitat de recompilar l'aplicació. [37]

Els models de decisió comencen amb la definició del model. Aquesta etapa implica estructurar el model amb nodes de decisions i dades d'entrada, establint quines decisions es deriven de quines condicions. Després, es crea l'element central, la taula de decisions. A partir d'aquesta, s'aplica el paradigma *if-then-else*, on cada fila de la taula representa una regla, amb condicions d'entrada i un resultat. Aquestes condicions s'escriuen amb el llenguatge FEEL (*Friendly Enough Expression Language*). Una vegada el model rep els *inputs*, aquests s'avaluen i es genera la sortida corresponent. L'avaluació pot ser de diferents tipus: seqüencial, amb multiplicitat de regles o prioritats o segons la política establerta (*hit policy*⁹). [39]

De forma resumida, la regla de decisions DMN rep dades com a entrada, amb el motor DMN, implementat amb Drools, s'avalua fila a fila si es compleixen aquestes i, en el moment que una fila coincideix amb les condicions, s'aplica la sortida. Amb això es retorna un resultat que s'utilitza per continuar amb el procés BPMN. [39]

PMML

PMML correspon a les sigles Llenguatge de marcatge per a la mineria de dades (*Predictive Model Markup Language*) i està desenvolupat pel *Data Mining Group* (DMG). [40] Es basa en XML i proporciona un estàndard obert que serveix per representar models predictius d'aprenentatge autònom entre diferents aplicacions. Els models es poden entrenar amb eines com *python*, *R* i *SAS*, entre d'altres, i poden intercanviar-se i executar-se en diferents entorns sense dependències específiques.

En el projecte, PMML s'ha utilitzat per integrar un model predictiu de risc dins del back-end. El model s'executa com a part d'una decisió DMN, gràcies al motor Kogito que ofereix una compatibilitat nativa amb aquest tipus de models.

⁹ Una *hit policy* especifica quantes regles d'una taula de decisions es poden satisfer i quantes de les regles satisfetes s'inclouen en el resultat de la decisió. [38]

API REST

El projecte s'ha dissenyat amb una arquitectura orientada a serveis que, mitjançant *endpoints* RESTful, ha permès comunicar el front-end i el back-end. Aquesta API segueix el patró CRUD (*Create, Read, Update, Delete*) i facilita l'intercanvi de dades amb el format JSON.

El back-end utilitza JAX-RS (Jakarta RESTful Web Services) per construir serveis REST, que a través de Quarkus, s'integra amb el mòdul *resteasy*.

9.3. Disseny de la base de dades

La base de dades utilitzada és PostgreSQL i emmagatzema tota la informació relativa a les sol·licituds de préstecs, les notificacions i l'historial d'accions de les sol·licituds associades al procés. A més, conté taules generades automàticament pel motor de processos Kogito per gestionar l'estat intern del procés BPMN.

Així doncs, hi ha dues categories de taules. Per un costat, les taules pròpies del domini del projecte i, per l'altre costat, les taules de suport a la gestió de processos BPMN i tasques.

Taules pròpies del domini del projecte

Aquestes taules estan dissenyades manualment per cobrir les necessitats de la lògica de negoci:

- ***Solicitudes***: conté la informació principal de cada petició de préstec i inclou també informació referent al client, obtinguda del microservei.
- ***Notificaciones***: registra les notificacions generades pel sistema durant el procés.
- ***Auditoria_solicitudes***: guarda un registre de totes les accions rellevants realitzades en cada sol·licitud.

Taules de suport a la gestió de processos BPMN i tasques

Aquestes taules es generen automàticament amb Kogito i serveixen per controlar i persistir informació del flux de processos i les tasques humanes:

- ***processes, processes_addons, processes_roles***: emmagatzemen informació dels processos en execució, les seves extensions i rols associats.
- ***definitions, definitions_nodes, definitions_annotations, definitions_metadata, definitions_nodes, definitions_nodes_metadata, definitions_roles***: contenen la definició del procés BPMN i els seus components.

- ***tasks, tasks_admin_groups, tasks_admin_users, tasks_excluded_users, tasks_potential_groups, tasks_potential_users***.: taules relacionades amb la gestió i assignació de tasques humanes dins del flux.
- ***jobs, kogito_data_cache, milestones***: suporten la persistència de l'estat, temporització i informació compartida dels processos.

Aquest és el diagrama de la base de dades, tenint en compte totes les taules:

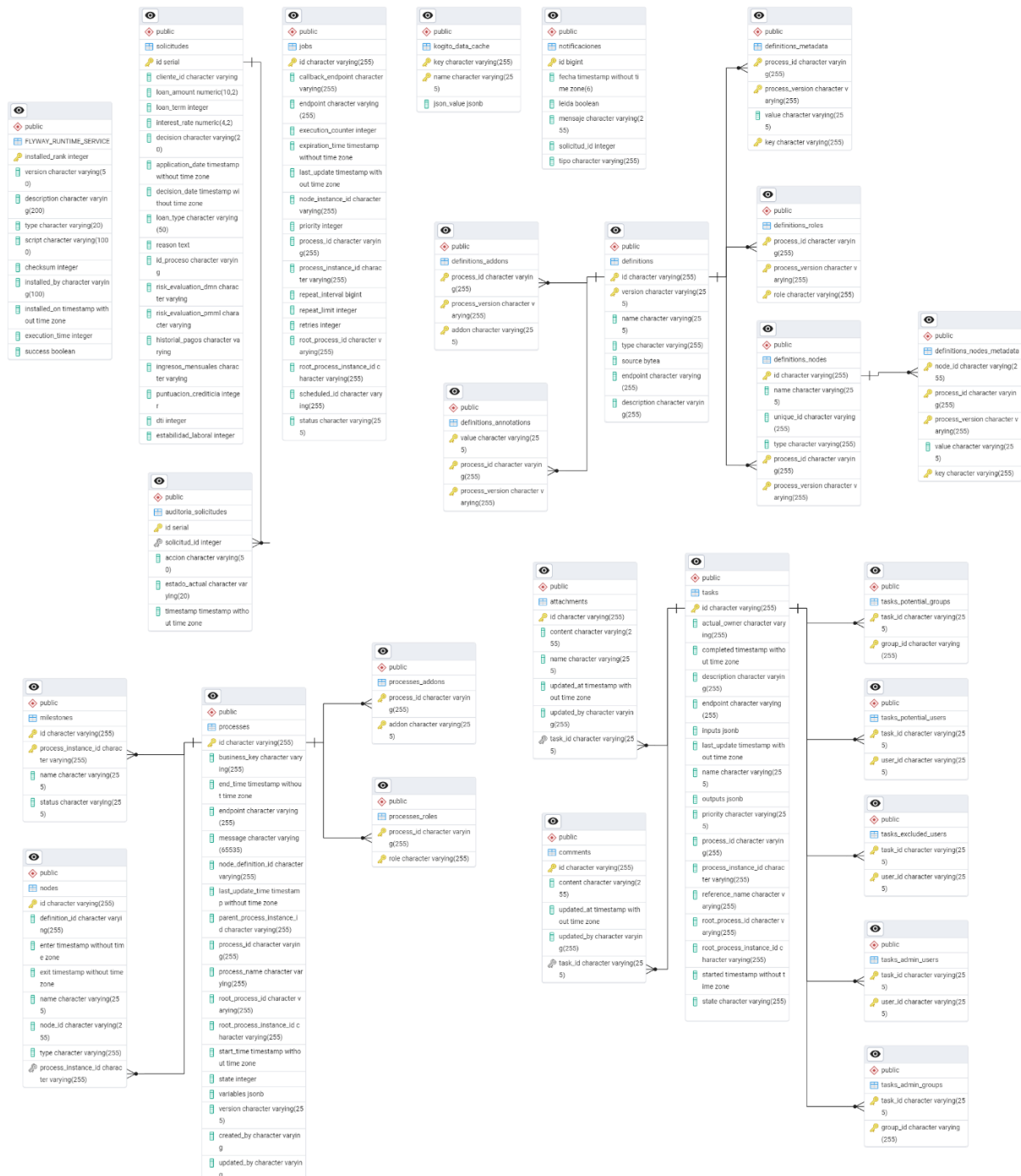


Figura 5. Diagrama de la base de dades
Font: elaboració pròpia

9.4. Disseny de la interfície

9.4.1. Estructura de la plataforma

La interfície d'usuari té l'objectiu de proporcionar una plataforma senzilla per visualitzar de forma intuïtiva i funcional les funcionalitats desenvolupades al back-end. L'estructura general de la plataforma es compon per una barra lateral que actua com un element de navegació principal entre les seccions Taulell, Registre de sol·licituds, Cerca de sol·licituds i Notificacions. Aquesta es troba a la part esquerra de totes les pantalles per permetre una navegació més ràpida i intuïtiva. A banda d'això, el flux d'usuari és coherent per poder registrar una nova sol·licitud, veure les sol·licituds agrupades per estats, consultar els detalls d'una sol·licitud en concret i accedir a les notificacions.

L'aplicació està desenvolupada amb Angular i utilitza components modulars. El disseny és *responsive*, de manera que es pot adaptar a diferents pantalles i així complir amb els objectius de disseny establerts: evitar sobrecàrrega d'informació i facilitar l'accés a la informació més rellevant.

9.4.2. Mapa navegacional entre les pantalles

L'aplicació està estructurada en diverses vistes que s'enllacen entre elles a través de rutes proporcionades per Angular. La figura 6 mostra el mapa navegacional entre les pantalles de la plataforma i aquestes s'expliquen a continuació.

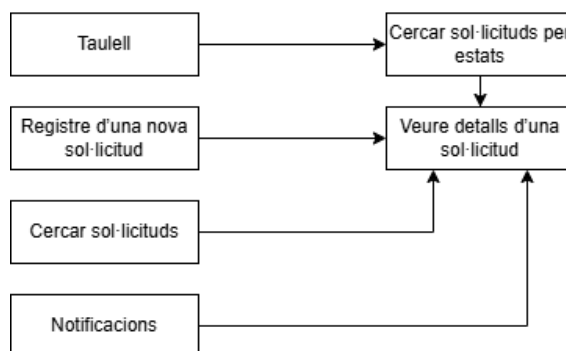


Figura 6. Mapa navegacional de les pantalles de la interfície
Font: elaboració pròpia

Taulell (Pantalla inicial)

És la vista principal de l'aplicació. Té accés a les estadístiques globals i als enllaços directes de les pantalles per visualitzar el llistat de sol·licituds en funció del seu estat: Aprovada, Rebutjada o Pendent.

Registre d'una nova sol·licitud

Aquesta pantalla presenta un formulari on s'han d'introduir les dades bàsiques necessàries per poder iniciar la petició i així determinar amb el flux si la sol·licitud per rebre un préstec hipotecari s'aprova o es rebutja. En el moment que es registra el formulari, s'inicia el flux BPMN del back-end.

Cercar sol·licituds

Conté una llista de totes les sol·licituds enregistrades al sistema. Per a cada sol·licitud es visualitza el seu identificador, l'acció en la qual es troba i la decisió presa. A banda d'això, es pot cercar una sol·licitud en particular a partir del seu identificador i es poden filtrar les sol·licituds en funció de la decisió i l'acció.

Cercar sol·licituds per estats

Aquesta pantalla és similar a la pantalla 'Cercar sol·licituds' però es diferencia d'aquesta perquè mostra directament sol·licituds Aprovades, Rebutjades o Pendents en funció del que s'hagi escollit des del taulell. De la mateixa manera que "Cercar sol·licituds", es pot cercar una sol·licitud a partir del seu identificador, però en aquest cas, no hi ha més filtratge aplicat.

Veure detalls d'una sol·licitud

És la vista on es poden veure tots els detalls d'una sol·licitud. És en aquesta pantalla on es completen les tasques humanes generades pel flux BPMN en el cas que sigui necessari.

Notificacions

La pantalla de notificacions mostra el llistat de notificacions que genera el sistema en les situacions establertes en el flux BPMN. Aquestes situacions poden ser per avisar a l'usuari de les tasques manuals que ha de completar o per informar sobre la decisió final presa d'una sol·licitud. Aquesta llista de notificacions conté de forma predeterminada les notificacions que no s'han llegit, però es poden filtrar per veure-les totes. D'altra banda, a partir de cada notificació, es pot accedir a la pantalla per veure els detalls de la sol·licitud en particular.

9.4.3. Mockups

Abans d'implementar les pantalles comentades en l'apartat anterior, es van desenvolupar uns *mockups* per tenir un disseny que servís com a guia per garantir que la interfície fos clara i orientada a les funcionalitats principals. Seguidament, es troben els dissenys.

En primer lloc, es troba la pantalla principal, el taulell. Aquesta mostra un resum visual de les sol·licituds agrupades per estats.



*Figura 7. Mockup pantalla Taulell
Font: elaboració pròpia*

En segon lloc, es troba la pantalla per registrar la sol·licitud. Les dades que inicialment es van definir que havia d'incloure el formulari són: l'identificador del client, l'import, el termini, la taxa i el tipus de préstec.

The image shows a web application mockup for 'REGISTRO DE NUEVA SOLICITUD'. The form has a white background with a dark blue header. It contains five input fields with labels and values: 'CLIENTE ID' (998670), 'MONTO DEL PRÉSTAMO' (152377€), 'PLAZO DEL PRÉSTAMO' (30 años), 'TASA DE INTERÉS' (2.4%), and 'TIPO DE PRÉSTAMO' (Hipotecario fijo). A dark blue 'Guardar' button is at the bottom.

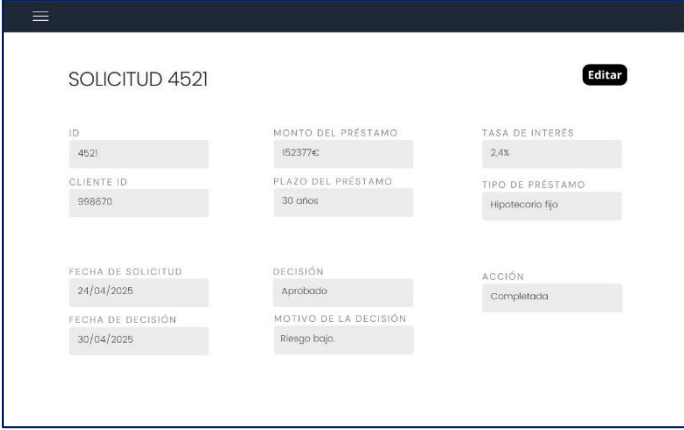
*Figura 8. Mockup pantalla Registre nova sol·licitud
Font: elaboració pròpia*

En tercer lloc, a les figures 9 i 10 es pot veure el llistat de sol·licituds i els detalls d'una en concret.



ID SOLICITUD	ACCIÓN	DECISIÓN/ESTADO
4522	Validación documentación	En proceso
4525	En evaluación PMML	En proceso
4526	En revisión manual	En proceso

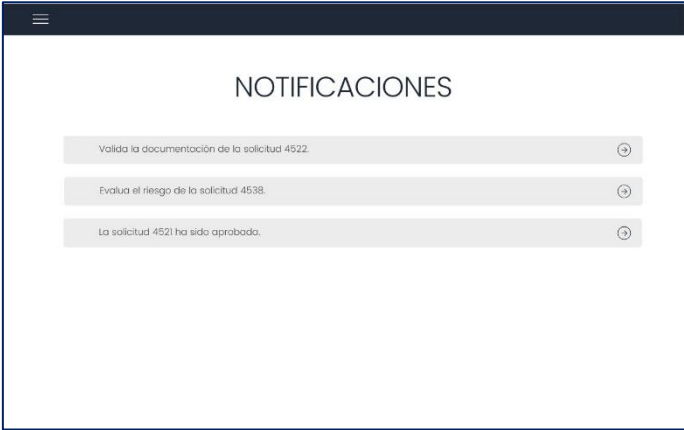
*Figura 9. Mockup pantalla Cercar sol·licitud per estat
Font: elaboració pròpia*



ID	MONTO DEL PRÉSTAMO	TASA DE INTERÉS
4521	152377€	2.4%
CLIENTE ID	PLAZO DEL PRÉSTAMO	TIPO DE PRÉSTAMO
998670	30 años	Hipotecario fijo
FECHA DE SOLICITUD	DECISIÓN	ACCIÓN
24/04/2025	Aprobado	Completada
FECHA DE DECISIÓN	MOTIVO DE LA DECISIÓN	
30/04/2025	Riesgo bajo.	

*Figura 10. Mockup pantalla Detalls de la sol·licitud
Font: elaboració pròpia*

Finalment, el darrer *mockup* correspon a la pantalla de notificaciones.



Valida la documentación de la solicitud 4522.	→
Evalua el riesgo de la solicitud 4538.	→
La solicitud 4521 ha sido aprobada.	→

*Figura 11. Mockup pantalla Notificacions
Font: elaboració pròpia*

9.5. Microservei

L'avaluació de les sol·licituds de préstecs requereix informació detallada sobre el perfil financer del client que demana la sol·licitud. Aquesta informació pot ser molt variada, però pel que fa a aquest projecte, s'ha determinat que són necessàries les dades de la puntuació creditícia, l'estabilitat laboral, els ingressos mensuals, el DTI i l'historial de pagaments.

Per obtenir aquesta informació, s'ha desenvolupat un microservei independent encarregat de retornar les dades del client a partir del seu identificador únic. Aquest servei funciona de manera paral·lela al back-end principal i s'hi accedeix a partir de peticions HTTP.

El microservei s'ha implementat amb Spring Boot, un *framework* de Java que simplifica la creació de serveis back-end de forma ràpida. Spring Boot permet crear APIs RESTful de forma simple, fer arrencades ràpides i permet tenir una autoconfiguració automàtica que redueix la necessitat de configurar manualment molts aspectes tècnics del projecte, com la connexió a base de dades, l'arrencada del servidor o la configuració de serveis REST. [41]

9.6. Diagrama BPMN d'actors i de l'orquestració del procés hipotecari

A la figura següent es presenta un diagrama BPMN que modela el procés global de validació i avaluació d'una sol·licitud de préstec hipotecari, destacant la interacció entre el back-end, el front-end i el microservei.

Aquest flux comença quan l'usuari omple el formulari al front-end, i finalitza amb la generació d'un resum i l'actualització de la decisió (aprovada o rebutjada).

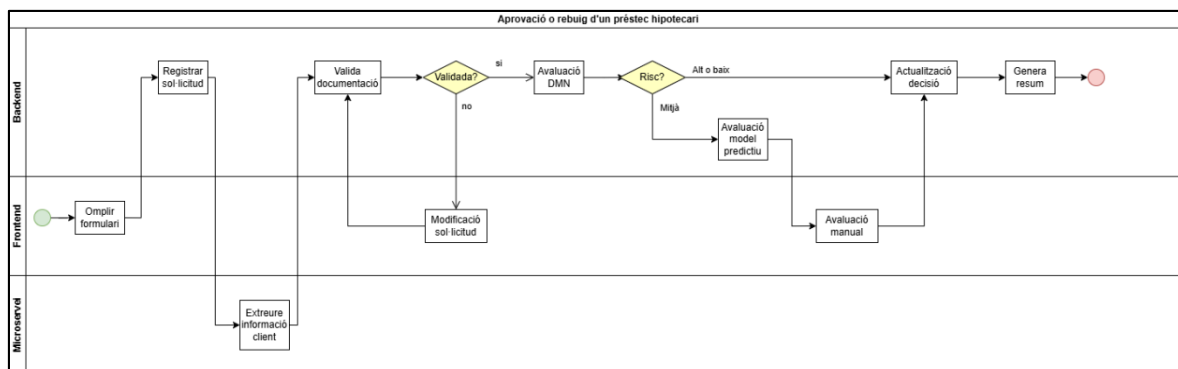


Figura 12. Diagrama BPMN d'orquestració del procés
Font: elaboració pròpia

9.7. Patrons de disseny utilitzats

Durant el desenvolupament del sistema, s'han implementat diferents patrons de disseny que assegurin una arquitectura clara i escalable. A continuació es detallen els patrons emprats.

Patró Model – View– Controller (MVC)

El patró MVC s'utilitza com a estructura global de tota l'aplicació i inclou tres elements: el model, la vista i el controlador. El model representa les dades del sistema, com ara *Solicitudes*, *Notificaciones* i *DTOs*. La vista implementada mostra informació relacionada amb les sol·licituds a través de la interfície. El controlador REST actua com a nexa entre el front-end i el back-end.

Patró Controlador – Servei – Repositori

Aquest patró de capes del back-end es basa a dividir l'aplicació en 3 capes per separar responsabilitats. D'una banda, el controlador rep i gestiona les peticions HTTP i les delega al servei corresponent. D'altra banda, el servei conté la lògica del codi i les regles de domini. Finalment, el repositori accedeix a la base de dades per fer consultes o persistir dades.

Aquest patró facilita el manteniment, ja que es pot reutilitzar codi i permet integrar tests unitaris de forma senzilla. Per exemple, en el cas de les sol·licituds, *SolicitudesController* delega la lògica a *SolicitudesService*, que inclou *SolicitudesServiceImpl*, i aquest utilitza *SolicitudesRepository* per accedir a les dades de la base de dades.

Patró Repository

El patró repositori s'utilitza per unir la capa lògica de negoci amb la base de dades i així el codi del servei no està vinculat directament a les dades. Així mateix, encapsula l'accés a les dades i proporciona una interfície clara per fer operacions CRUD o consultes específiques. Aquest patró s'aplica en els repositoris del projecte, donat que implementen *PanacheRepository* de Quarkus, una funcionalitat que permet accedir i gestionar entitats i ofereix mètodes predefinits com *findById()*.

Patró Singleton

El patró Singleton assegura que una classe només tingui una única instància durant el cicle de vida de l'aplicació i aquesta pugui ser accessible de forma global. En el context de desenvolupament amb Quarkus, aquest patró es gestiona automàticament mitjançant el contenidor de dependències.

En el projecte, aquest patró s'aplica als serveis del back-end, com per exemple a *SolicitudesServiceImpl*, que està anotada amb *@ApplicationScoped*, la qual cosa garanteix que només s'ha d'instanciar una vegada i es comparteix al llarg de tota l'aplicació. Així doncs, *SolicitudesServiceImpl* encapsula tota la lògica relacionada amb la gestió de sol·licituds en una instància injectada al controlador mitjançant *@Inject*.

Patró Injecció de dependències (CDI)

El patró d'injecció de dependències és un principi fonamental en el desenvolupament d'aplicacions perquè permet que els components no creïn directament les seves dependències, sinó que aquestes són proporcionades pel contenidor d'injecció. Les anotacions necessàries perquè es porti a terme són: *@Inject* per Quarkus i *@Autowired* per Spring.

Patró Factory

El patró Factory és un patró de creació per desplegar objectes en una entitat centralitzada, fet que permet encapsular la lògica de construcció i retornar instàncies preparades. En el cas particular del projecte, aquest patró queda reflectit en el funcionament del contenidor d'injecció de dependències en el moment que s'anota *@ApplicationScoped*, *@Service* o *@Repository*, que indica que s'han de crear les instàncies d'aquesta classe només quan siguin necessàries.

Patró Builder

El patró Builder es basa a construir DTOs a partir d'objectes del domini a l'hora de crear respostes API. En el projecte, s'ha utilitzat l'anotació *@Builder* de Lombok per generar automàticament un constructor i així crear instàncies d'objectes.

A banda d'això, Spring Boot també fa servir aquest patró per crear respostes HTTP amb *ResponseEntity*, creant capçaleres, cossos i codis d'estat de forma fluida.

Patró Proxy

Aquest patró s'utilitza en escenaris de programació orientada a aspectes. Els mètodes del projecte que es basen en aquest patró estan anotats amb *@Transactional*, que creen un proxy dinàmic per cadascun d'aquests mètodes. Aquest Proxy s'encarrega de gestionar l'obertura i el tancament de la transacció a la base de dades. D'aquesta manera, es garanteix que qualsevol operació d'escriptura només sigui efectiva si es completa correctament el mètode i, en cas d'error, es faci *rollback*.

9.8. Principis SOLID

Per desenvolupar el sistema, s'han considerat els principis SOLID per tenir un sistema que compleixi bones pràctiques. Els principis SOLID són una guia per al disseny orientat a objectes que serveix per garantir que el codi és modular, mantenible i desacoblat. [42]

S: Principi de Responsabilitat Única

Cada classe del projecte té una responsabilitat única. Per exemple, la classe *NotificacionesServiceImpl* s'encarrega exclusivament de la lògica relacionada amb les notificacions. D'altra banda, la classe *NotificacionesController* genera peticions HTTP i *NotificacionesRepository* encapsula les operacions de persistència a la base de dades.

O: Obert per extensió, tancat per modificació

Aquest principi indica que les funcionalitats es poden ampliar sense modificar el codi existent. Per exemple, les interfícies i els DTOs com *AuditoriaSolicitudesService* permeten implementar nova lògica sense eliminar el codi existent.

L: Principi de substitució de Liskov

El principi de substitució de Liskov garanteix flexibilitat i la possibilitat de canviar comportaments per evolució o *testing*. *SolicitudesServiceImpl* implementa la interfície *SolicitudesService*, però aquesta podria ser substituïda per una altra implementació sense afectar el comportament dels controladors.

I: Principi de segregació d'interfícies

S'han definit interfícies específiques per definir els mètodes rellevants de cada model, és a dir, *SolicitudesService* defineix els mètodes per gestionar les sol·licituds. De la mateixa manera, els serveis *NotificacionesService* i *AuditoriaSolicitudesService* tenen les seves pròpies interfícies adaptades a les seves funcionalitats particulars.

D: Principi d'inversió de dependències

El codi no depèn d'implementacions concretes sinó que conté abstraccions. Dit amb altres paraules, els controladors depenen de serveis definits i no directament d'implementacions. A més a més, en utilitzar *@Inject*, Quarkus instància directament la implementació adequada i això afavoreix el desacoblament. El desacoblament es refereix a reduir la dependència entre diversos components d'un sistema.

10. IMPLEMENTACIÓ

10.1. Estructura del codi i mòduls principals

10.1.1. Back-end principal

El back-end s'ha estructurat de tal manera que es garanteixi la separació de responsabilitats i es faciliti el manteniment del codi, tenint en compte que l'arquitectura es basa en Quarkus i Jakarta EE. A continuació es descriu aquesta estructura:

Src/main/java

Conté el codi principal de l'aplicació i està dividit en diversos paquets:

- **Controller:** conté les classes encarregades de gestionar les peticions HTTP. Cada classe defineix un *endpoint* REST mitjançant les anotacions de Jakarta REST, com són *@Path*, *@GET*, *@POST*, *@PUT* i *@DELETE*. A banda d'això, el format de la resposta es defineix com a *@Produces(MediaType.APPLICATION_JSON)*.
- **Service i service.impl:** conté les interfícies i les implementacions que encapsulen la lògica de negoci. Cada servei es defineix com a *@ApplicationScoped*, indicant que hi ha només una instància de cada un a nivell d'aplicació.
- **Repositori:** inclou les classes que gestionen l'accés a la base de dades. Totes les classes que actuen com a repositoris implementen la interfície *PanacheRepository*.
- **Model:** conté les entitats del domini, com *Solicitudes*, *Notificaciones* i *AuditoriaSolicitudes*. Aquestes classes estan anotades amb *@Entity* de Jakarta Persistence per poder persistir-se a la base de dades PostgreSQL.
- **DTO:** conté els DTOs que encapsulen les dades que es transmeten entre capes.

Src/main/java/Resources

La carpeta *Resources* conté els fitxers BPMN i DMN, explicats en els subapartats següents, i l'arxiu de configuració *application.properties*.

Aquest fitxer és l'arxiu central de configuració de Quarkus i conté els paràmetres essencials pel funcionament del sistema. Pel que fa a la persistència, la base de dades PostgreSQL queda configurada amb *quarkus.datasource.** i la generació automàtica d'un esquema es fa gràcies a *quarkus.hibernate-orm.database.generation = update*.

A més, les migracions amb Flyway estan habilitades i les rutes URL per utilitzar els serveis de Kogito també estan definides. Aquest arxiu també permet tenir una exposició permanent de la

interfície Swagger i ofereix al back-end connectar-se des del front-end, a partir de la configuració de CORS.

Src/test/java

Conté les proves del sistema, incloent-hi les proves unitàries per validar el bon funcionament dels mètodes implementats.

Pom.xml

Aquest arxiu és l'element central de configuració del projecte Maven, és a dir, defineix les dependències, versions, *plugins* i propietats necessàries per compilar, construir i executar l'aplicació. En particular, a les dependències s'importen els BOMs¹⁰ de Quarkus i Kogito, assegurant que les versions entre les dependències és coherent. S'estableix la versió 3.8.6 de Quarkus, 10.0.0 de Kogito i 17 de Java.

Al *pom.xml* del projecte només s'inclouen els artefactes de Kogito, i no els de BAMOE, ja que es desenvolupa un flux concret basat en Kogito. Tot i això, BAMOE podria utilitzar components de Kogito internament.

El projecte incorpora diversos mòduls essencials:

- **Quarkus Core:** proporciona bases per construir aplicacions, utilitzant serveis web RESTful usant JAX-RS, oferint suport per gestionar dependències i cicles de vida dels components i generant automàticament *endpoints* REST en format OpenAPI (*quarkus-resteasy*, *quarkus-arc*, *quarkus-smallrye-openapi*).
- **Persistència:** inclou les dependències per gestionar la base de dades i el model de dades (*quarkus-hibernate-orm*, *quarkus-jdbc-postgresql*).
- **Regles i processos:** serveixen per gestionar processos empresarials i la presa de decisions (*jbpm-with-drools-quarkus*, *kie-addons-quarkus-process-management*, *kogito-addons-quarkus-data-index-postgresql*).
- **Testing:** dependències necessàries per fer tests unitaris i de servei REST (*quarkus-junit5*, *rest-assured*, *kogito-quarkus-test-utils*).
- **Altres:** per simplificar el codi JAVA, es generen automàticament getters, setters i altres components amb *Lombok*.

¹⁰ Tipus d'arxiu POM (*Project Object Model*) que conté la llista de dependències i les versions corresponents pel projecte. [43]

Pel que fa al procés de construcció (*build*), hi ha els següents *plugins*:

- *quarkus-maven-plugin*: compila i empaqueta l'aplicació com un executable natiu o JAR.
- *maven-compiler-plugin*: assegura que es compili amb la versió correcta de Java i integra *Lombok* per al processament d'anotacions.

10.1.2. Diagrama de classes

La següent figura mostra el diagrama de classes implementat.

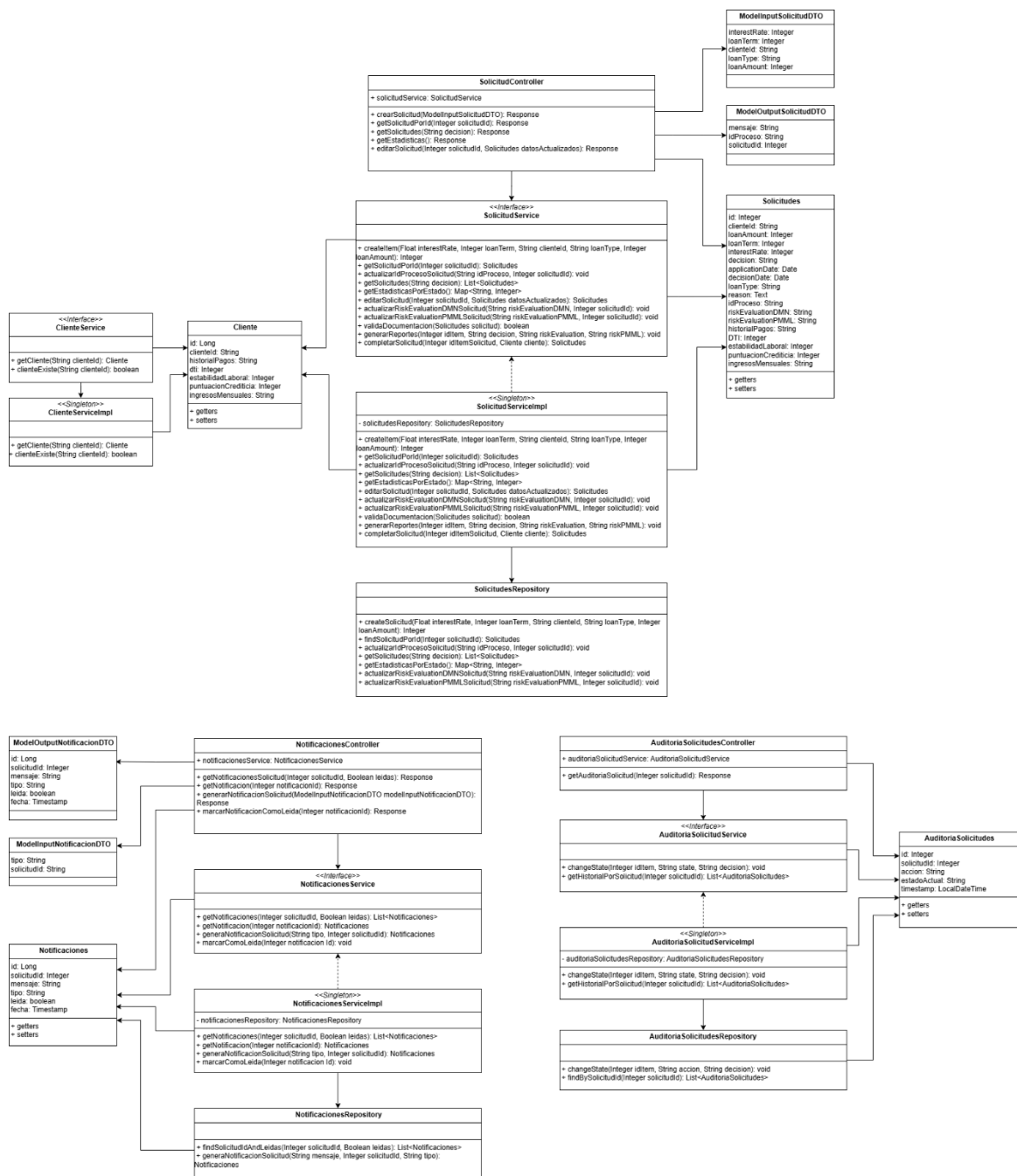


Figura 13. Diagrama de classes
Font: elaboració pròpia

10.1.3. Microservei

Parlant del microservei, aquest funciona de manera independent al back-end principal, té la seva pròpia estructura, amb controladors, serveis i models, i exposa una API REST que retorna la informació necessària dels clients. El codi d'aquest microservei es troba en un repositori separat.

10.2. Implementació del procés BPMN

Tot el projecte es basa en l'orquestració del procés BPMN que actua com a eix de la lògica del sistema. A continuació es mostra aquest flux BPMN implementat, que defineix si es pot concedir o no un préstec hipotecari. En el cas de voler veure'l amb més detall, anar a l'annex 1.

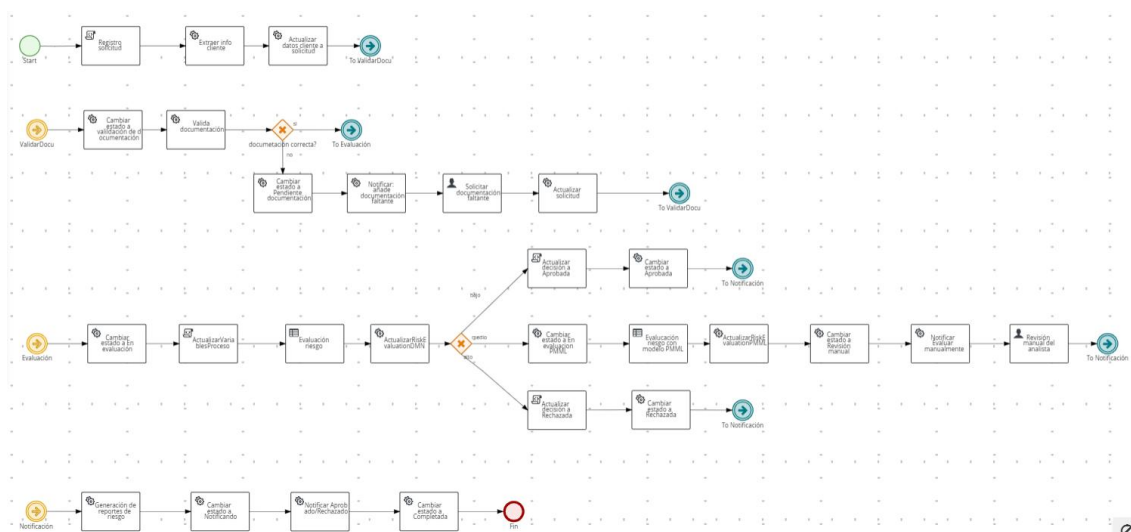


Figura 14. Flux BPMN del préstec hipotecari
Font: elaboració pròpia

Aquest procés modela tot el cicle de vida d'una sol·licitud de préstec hipotecari, des del moment en què l'usuari registra la sol·licitud fins a la seva resolució. En el moment que la sol·licitud s'ha enregistrat, es fa una crida al microservei per obtenir dades del client i després es valida aquesta. Si la documentació és correcta es passa a la fase d'avaluació, però, en cas contrari, el flux agafa el camí per sol·licitar a un analista del banc que modifiqui aquestes dades de forma manual. Una vegada les dades han estat actualitzades, es torna a fer la validació. L'avaluació de la petició es porta a terme a partir d'un model de decisions que retorna el risc d'aprovar aquesta. Pot retornar tres valors: Baix, Mitjà, Alt. En el cas que el risc sigui baix, la decisió passa a ser automàticament Aprobada. De la mateixa manera, si el risc és alt, la sol·licitud és Rebutjada. És pel cas en què el risc és mitjà que el procés passa pel camí on es duu a terme una altra avaluació del risc però aquesta vegada amb un model predictiu. Després de passar per aquest model, es demana a un

analista que manualment triï la decisió final. Una vegada la decisió s'ha pres, s'informa i es marca la sol·licitud com a completada.

En aquest procés es defineixen variables globals que s'utilitzen al llarg de les tasques per tal d'obtenir el resultat final. Entre les variables globals més destacades definides es troben: *puntuacionCrediticia*, *DTI*, *historialPagos*, *estabilidadMensual*, *interestRate*, *riskEvaluation*, *riskEvaluationPMML*, *clienteId*, *idItemSolicitud* i *decision*. Aquestes permeten passar informació entre tasques i prendre decisions dins del flux.

Pel que fa a les tasques que hi ha implementades, es poden trobar dels següents tipus:

1. Service tasks

Es tracta de tasques automàtiques que criden a les funcions implementades als serveis. Així doncs, s'ha de tenir en compte que s'han de passar els paràmetres requerits d'entrada i s'ha de guardar la sortida correctament en el cas que es retornin un o més valors. Aquestes tasques tenen un engranatge dibuixat.

D'entre aquest tipus de tasques, es troba una per extreure la informació del client, cridant a la funció que crida al servei extern per obtenir la informació del client. També totes les tasques per canviar l'estat de la sol·licitud, validar la documentació, crear notificacions i actualitzar les variables del sistema i de la base de dades són d'aquest tipus.

2. User tasks

Són tasques que representen punts del procés on ha d'intervenir una persona per realitzar-la. En el moment que s'executa, el flux queda paralitzat fins que queda completada.

En particular, el procés del préstec hipotecari conté dues *user tasks*. Per una banda, en el cas que les dades del client siguin incorrectes, un usuari ha de modificar-les manualment. Per l'altra banda, si es porta a terme una avaluació del risc amb el model predictiu, per assegurar un bon resultat de la decisió, un usuari ha de comprovar manualment els resultats obtinguts en els models de decisió i determinar si la petició es pot aprovar o no.

3. Script tasks

Els *script tasks* executen un fragment de codi dins del procés, sense necessitat de cridar a serveis externs ni intervenció humana. S'ha utilitzat en alguns casos per inicialitzar o actualitzar variables globals (*Registro solicitud*, *Actualizar variables proceso*, *Actualizar decisión*).

4. Decision tasks

Són tasques que invoquen a un model de decisió DMN, implementat en un altre fitxer amb extensió *.dmn*, per aplicar regles de negoci de forma declarativa.

El procés inclou dues *decision tasks*, ambdues per determinar si el risc de la sol·licitud és baix, mitjà o alt, però una utilitza un model de decisions que conté una taula de decisions i l'altre un model predictiu en format PMML. Aquests models de decisió s'expliquen amb més detall en els subapartats següents.

Per executar el codi dins de Kogito, aquest transforma el procés en un servei REST desplegable i genera automàticament *endpoints* per iniciar i interactuar amb aquest. A banda d'això, el flux emet i escolta esdeveniments que permeten coordinar el *back-end* amb l'estat intern del procés per actualitzar la base de dades.

Finalment, cal remarcar que s'han seguit recomanacions de bones pràctiques a l'hora de desenvolupar el flux per tenir simplicitat en aquest, noms clars i bona gestió de bifurcacions. [44]

10.2.2. Model de decisions DMN

El model de decisions DMN s'ha creat amb la finalitat de determinar de forma ràpida el risc que suposa aprovar una sol·licitud per aconseguir un préstec hipotecari. El model es compon principalment per un node anomenat *riskEvaluation*, el qual representa la decisió final sobre el nivell del risc. Aquesta decisió es calcula a partir de cinc *inputs* que es representen com a nodes connectats a la taula de decisions. Aquestes són les variables d'entrada:

1. DTI (Debt-to-Income): és un percentatge que representa la relació entre el deute mensual i els ingressos mensuals del client, és a dir, indica quants diners gasta la persona en pagar deutes en vers a quants diners entra al seu grup familiar. [45]
2. Puntuació creditícia: és un valor numèric entre 0 i 850 que mesura la solvència financera del client. És una eina utilitzada pels bancs i altres institucions financeres per analitzar diversos aspectes econòmics i financers relacionats amb el sol·licitant d'un préstec. Determina la probabilitat que aquesta persona compleixi amb les seves obligacions de pagament de manera completa i puntual.[46]
3. Estabilitat laboral: és el nombre d'anys d'estabilitat en el lloc de treball.
4. Historial de pagaments: determina el comportament de pagaments anteriors i pot comprendre els següents valors: "Puntuales", "Tardíos", "Frecuentes".

5. Ingressos mensuales: és una qualificació qualitativa de la capacitat econòmica del client. Per considerar-se un valor vàlid ha de comprendre algun d'aquests valors: "Insuficientes", "Justos", "Suficientes".

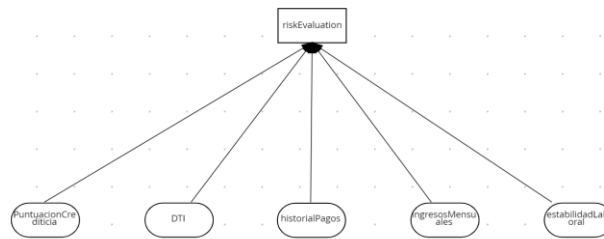


Figura 15. Model de decisió DMN
Font: elaboració pròpia

La taula de decisió implementada defineix les regles de negoci basant-se en el valor dels paràmetres d'entrada. Cada fila representa una regla i se segueix una política *First Hit*, de manera que quan es compleix una de les condicions definides, s'aplica la sortida corresponent sense continuar avaluant les següents regles.

F	DTI (number)	PuntuacionCrediticia (number)	estabilidadLaboral (number)	historialPagos (string)	IngresosMensuales (string)	riskEvaluation (string)
1	>50	-	-	-	-	"Alto"
2	-	<600	-	-	-	"Alto"
3	-	-	<1	-	-	"Alto"
4	-	-	-	"Tardios"	-	"Alto"
5	-	-	-	-	"Insuficientes"	"Alto"
6	<40	>750	>=1	"Puntuales"	-	"Bajo"
7	<40	[700..750]	>=1	"Puntuales"	-	"Bajo"
8	<40	[700..750]	>=1	"Frecuentes"	-	"Bajo"
9	-	-	-	-	-	"Medio"

Figura 16. Taula de decisions DMN
Font: elaboració pròpia

A partir de la taula presentada a la figura 16, es poden veure les diferents regles que determinen si el risc és alt, mitjà o baix. Es considera que la petició s'ha de rebutjar quan s'assoleixen valors extrems pels *inputs*, de manera que el risc serà alt. El cas del risc baix es dona en tres casos en particular i la resta es considera risc mitjà.

Aquest model està integrat dins del procés BPMN com una tasca de decisió (*decision task*) i s'executa automàticament. El procés s'encarrega de passar els *inputs* com a variables globals i Kogito executa la taula de decisions.

10.2.3. Model predictiu PMML

L'objectiu de desenvolupar un model predictiu és afegir una altra avaluació extra del risc creditici en el procés pels casos on el risc definit pel model de decisions és mitjà. El model es desenvolupa i s'exporta en format PMML (*Predictive Model Markup Language*), que correspon al llenguatge ideal per integrar el model a la plataforma BAMOE. La classificació del risc és de tres nivells: “Bajo”, “Medio”, “Alto”. Per aconseguir implementar el model s'han seguit els següents passos:

1. Definició de l'entorn

L'entorn és una eina essencial que s'ha de definir per poder desenvolupar i entrenar el model de *Machine Learning*, exportar-lo a *pmml* i integrar-lo al projecte. L'entorn de desenvolupament definit utilitza la versió 3.10 de *python* com a llenguatge d'entrenament. El model s'entrena amb *scikit-learn*, una llibreria de *python* que compta amb algoritmes de regressió, classificació i *clustering* i és compatible amb *pandas* i *numpy*, que són biblioteques també de *python* que serveixen per al processament de dades i pel càlcul científic respectivament. El model s'exporta a *.pmml* a partir de *sklearn2pmml* que té com a requisit utilitzar mínim Java 8.

Com a informació extra, s'ha necessitat Anaconda per crear l'entorn amb les característiques detallades anteriorment.

2. Definició del *dataset*

Una de les fases més importants en el moment de crear un model predictiu és la preparació de les dades que s'utilitzaran per al seu entrenament. Així doncs, s'ha fet un procés de *feature engineering* per garantir la qualitat de les dades. Aquestes dades s'han classificat en característiques numèriques i categòriques, amb l'objectiu d'aplicar el preprocessament adequat a cadascuna d'elles durant la fase d'entrenament.

Les variables categòriques són:

- *loan_type*: tipus de préstec (hipotecari, personal, cotxe).
- *historial_pagos*: comportament en els pagaments previs (Puntuales, Tardíos, Frecuentes).
- *ingresos_mensuales*: estimació de la capacitat d'ingrés (Suficientes, Insuficientes, Justos).

Les variables numèriques són:

- *loan_term*: termini del préstec (12, 24 o 36 mesos).
- *loan_amount*: quantitat total sol·licitada.

- `interest_rate`: interès associat al préstec.
- `puntuacion_crediticia`: puntuació creditícia del sol·licitant (entre 0 i 850).
- `diti`: ràtio de deute sobre ingressos (Debt-To-Income).
- `estabilidad_laboral`: anys de permanència en la feina actual.

La variable que es vol predir és *risk_evaluation_pmml*, la qual representa la classificació final del risc de la sol·licitud de préstec.

Una vegada definides les variables, s'ha implementat un script de *python* per generar aquestes dades i s'han guardat en un arxiu en format *.csv*, que s'utilitzarà en la pròxima fase.

3. Desenvolupament i entrenament del model

Després de generar el conjunt de dades sintètic, s'ha desenvolupat el model predictiu. Aquesta etapa inclou la lectura del *dataset* i el seu preprocessament, l'entrenament del model i la seva exportació en format PMML.

En primer lloc, es llegeix el fitxer creat en format *.csv* i es codifica la variable objectiu: *risk_evaluation_pmml*. Aquesta originalment era d'estil categòric, però s'ha transformat a valors numèrics, assignant el 0 al risc Baix, l'1 al risc Mitjà i el 2 al risc Alt.

En segon lloc, les dades es divideixen en dues categories. D'una banda, les columnes categòriques, que s'han codificat mitjançant *One-Hot Encoding*, i, d'altra banda, a les columnes numèriques, s'aplica *StandardScaler (Z-Score Standardization)*, per normalitzar cada atribut amb mitjana 0 i desviació estàndard d'1.

En tercer lloc, per l'entrenament del model, s'ha optat per un *Random Forest Classifier*. Aquest es tracta d'un algorisme basat en arbres de decisió que destaca per la seva robustesa i la gran capacitat de precisió en problemes de classificació amb dades mixtes. Aquest model construeix múltiples arbres de decisió i combina els resultats per obtenir una classificació precisa i estable.

En darrer lloc, el model entrenat es guarda en format PMML i amb això s'aconsegueix que tingui el format correcte per ser integrat en una taula DMN del BPMN.

4. Validació del model

Un cop s'ha entrenat el model predictiu, s'ha fet una validació per avaluar la seva eficàcia. El conjunt de dades s'ha dividit en dues parts per tenir diferents *subsets* de dades a l'hora d'entrenar i testejar el model.

Les mètriques que s'han considerat per validar el model són:

- *Accuracy*: Relació entre les prediccions correctes i el nombre total de prediccions.
- *Precision*: Proporció de positius veritables entre positius previstos.
- *Recall*: Proporció de positius veritables entre positius reals.
- *F1-Score*: Mitjana harmònica de *precision* i *recall*.
- *Confusion matrix*: Taula que resumeix el rendiment d'un model de classificació mostrant el recompte de positius i negatius veritables i falsos.

El model *RandomForestClassifier* ha demostrat que és més robust que altres algoritmes, com la regressió lineal o logística. Amb el *dataset* equilibrat s'ha assolit un *accuracy* global del 96% i la següent *confusion matrix*:

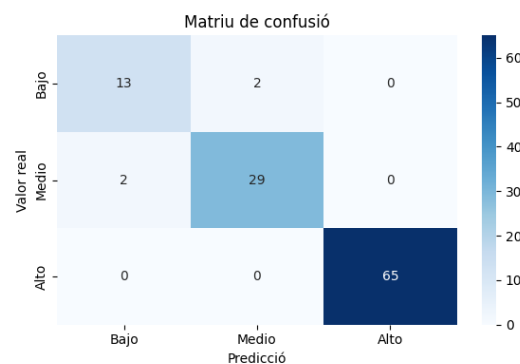


Figura 17. Matriu de confusió
Font: elaboració pròpia

El model mostra una precisió molt alta. La major part dels valors reals han estat classificats correctament, especialment la classe "Alto", amb zero errors. Només hi ha petits errors entre "Bajo" i "Medio", la qual cosa és esperable en classes amb certa proximitat.

5. Integració al BPMN

El model predictiu, gràcies a guardar-se amb format PMML, es pot integrar de manera simple al BPMN. En el flux s'ha de crear una tasca del tipus de decisions (DMN) i aquesta definir-la amb una funció PMML. A més a més, cal incloure el model dins del DMN i després fer la crida corresponent amb les dades d'entrada apropiades.

10.3. Desenvolupament de la interfície d'usuari

La interfície d'usuari té la intenció de permetre a l'usuari visualitzar de forma clara les funcionalitats implementades al flux BPMN i poder seguir el procés de decisió. L'objectiu principal no és oferir una experiència de l'usuari final exhaustiva, sinó permetre provar i verificar les diferents funcionalitats desenvolupades al sistema. És a partir d'aquest enfocament que el front-end ha facilitat el testeig integrat de l'API REST i la comprovació en temps real de l'execució de fluxos BPMN i les regles de negoci DMN i PMML. Així doncs, la plataforma permet registrar, cercar i llistar sol·licituds, visualitzar-les detalladament i veure les notificacions que genera el sistema.

El front-end s'ha implementat amb el *framework Angular* que estructura l'aplicació en components modulars i incorpora una biblioteca de components UI per tenir un disseny accessible i *responsive*. La definició de l'estat visual s'ha fet amb HTML i CSS.

L'aplicació Angular es comunica amb el back-end a través de serveis del propi *framework* que consumeixen *endpoints* REST mitjançant el mòdul HttpClient. Aquest utilitza observables per gestionar les respostes de manera asíncrona i utilitza el format JSON per fer l'intercanvi de dades.

A continuació es mostren les pantalles de les funcionalitats implementades tenint en compte els *mockups* inicials.

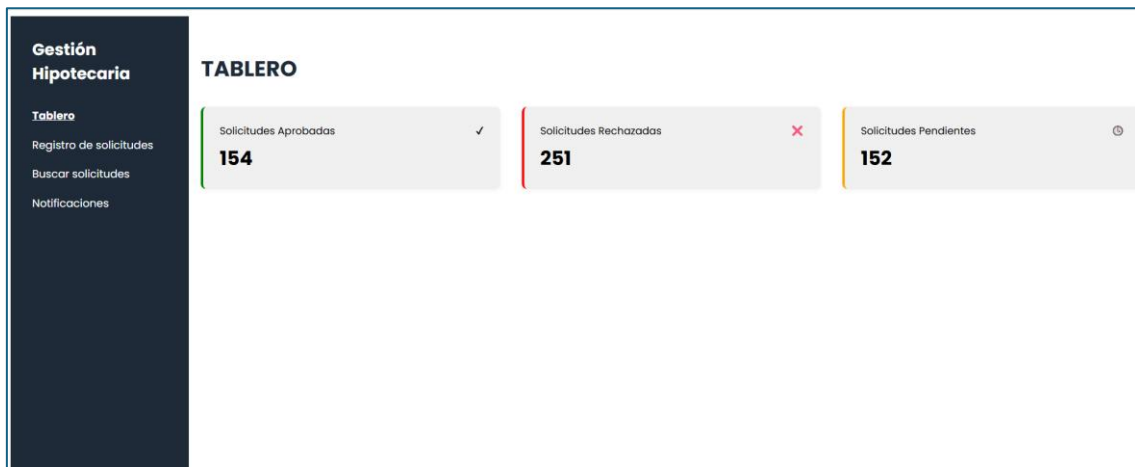


Figura 18. Pantalla Taulell
Font: elaboració pròpia

Gestión Hipotecaria

Tablero

Registro de solicitudes

Buscar solicitudes

Notificaciones

REGISTRO DE NUEVA SOLICITUD

CLIENTE ID

ID del cliente

IMPORTE DEL PRÉSTAMO

Monto del préstamo

PLAZO DEL PRÉSTAMO

Plazo del préstamo

TASA DE INTERÉS

Tasa de interés

TIPO DE PRÉSTAMO

Tipo de préstamo

Guardar

Cancelar

Figura 19. Pantalla Registre nova sol·licitud
Font: elaboració pròpia

Gestión Hipotecaria

Tablero

Registro de solicitudes

Buscar solicitudes

Notificaciones

← Volver

SOLICITUDES PENDIENTES

Buscar por identificador de la solicitud

ID SOLICITUD	DECISIÓN	ACCIÓN
4102	Pendiente	Pendiente documentación →
4101	Pendiente	En revisión manual →

Figura 20. Pantalla Cercar sol·licituds Pendants
Font: elaboració pròpia

Gestión Hipotecaria

Tablero

Registro de solicitudes

Buscar solicitudes

Notificaciones

BUSCAR SOLICITUDES

Buscar por identificador

En revisión manual

Pendientes

ID SOLICITUD	DECISIÓN	ACCIÓN
4101	Pendiente	En revisión manual →
4587	Pendiente	En revisión manual →
4377	Pendiente	En revisión manual →
4371	Pendiente	En revisión manual →
4356	Pendiente	En revisión manual →
4365	Pendiente	En revisión manual →
4216	Pendiente	En revisión manual →
4208	Pendiente	En revisión manual →

Figura 21. Pantalla Cercar sol·licituds
Font: elaboració pròpia



Figura 22. Pantalla Detalls de la sol·licitud
Font: elaboració pròpia

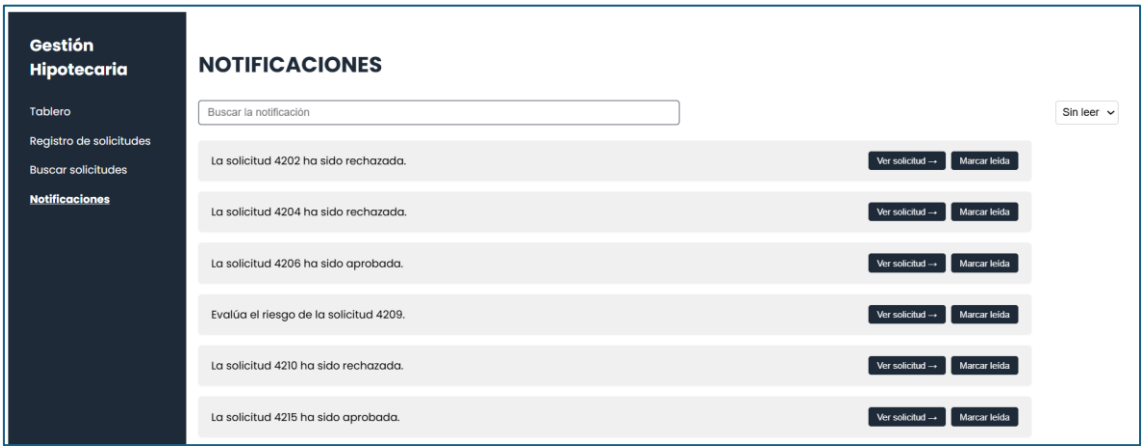


Figura 23. Pantalla Notificacions
Font: elaboració pròpia

10.4. Gestió d'errors

Per tal d'assegurar que el funcionament del codi sigui bo i consistent, s'han implementat mecanismes per capturar i tractar a temps els errors que sorgeixen durant l'execució del projecte.

A partir de bloc *try-catch*, els controladors REST gestionen els errors retornant respostes HTTP adequades segons el tipus d'aquest. Existeixen diferents tipus d'errors i es gestionen de la següent manera:

- Errors de validació o paràmetres invàlids: es controlen amb les excepcions de *BadRequestException* que retornen un codi 400 (*Bad Request*) amb el missatge explicatiu sobre l'error, com per exemple: "La notificació notificacionId no existe."
- Errors quan no es troben entitats: en aquest cas es llança l'excepció *EntityNotFoundException*, retornant 404 (*Not Found*).
- Errors inesperats i excepcions no controlades: es tracta d'excepcions amb codi 500 (*Internal Server Error*) o 502 (*Bad Gateway*) que mostren un missatge genèric que depèn del context en què s'ha provocat l'error.

A banda d'això, les anotacions *@Transactional* que hi ha als mètodes, assegura que si hi ha un error dins d'una operació de persistència, es faci *rollback* automàticament per evitar inconsistència de dades.

10.5. Eines i entorns de desenvolupament

El projecte s'ha desenvolupat a partir d'un conjunt d'eines i entorns que han permès implementar-lo i fer les proves adients per comprovar les seves funcionalitats.

10.5.1. Repositoris de codi

El treball s'estructura en tres repositoris privats independents, cadascun centrat en una part del sistema. Aquests es troben a GitHub i es gestionen amb controls de versions de Git. En tots els projectes de GitHub, les branques tenen la mateixa lògica, considerant la branca *master* com a principal i creant diferents subbranques (*dev*) per implementar funcionalitats específiques. Aquests són els repositoris:

- Repositori del back-end principal (<https://github.com/juliaTena/tfg-back-prestamo-hipotecario.git>): conté la lògica del sistema i està desenvolupat amb Quarkus i Kogito.
- Repositori del front-end (<https://github.com/JuliaTenaDomingo/prestamo-hipotecario-front.git>): fa referència al codi de la interfície d'usuari, programat amb Angular.

- Repositori del microservei (<https://github.com/JuliaTenaDomingo/tfg-cliente-microservicio.git>): és el repositori que funciona com a servei extern i està implementat amb Spring Boot.

10.5.2. IDEs utilitzats

El desenvolupament s'ha portat a terme utilitzant diferents IDEs en funció de la tecnologia necessitada. D'una banda, s'ha utilitzat IntelliJ IDEA per al back-end de Quarkus, el front-end d'Angular i pel microservei Spring Boot. D'altra banda, per treballar amb els models BPMN i DMN s'ha utilitzat Visual Studio Code, utilitzant l'extensió *Kogito Developer Tools*, per crear, editar i validar gràficament aquests processos i taules de decisions. A banda d'això, s'han emprat altres eines complementàries com Postman i Swagger UI per provar els *endpoints* REST de manera visual i ràpida.

10.5.3. Entorns d'execució

El sistema s'ha desenvolupat i executat en un entorn local. Quarkus s'ha fet servir pel back-end principal, Kogito per desplegar els processos BPMN i regles DMN, Spring Boot pel microservei de consulta dels clients i PostgreSQL com a sistema de dades relacional. Cal remarcar que gràcies a l'arquitectura modular i els components utilitzats, no ha calgut desplegar contenidors Docker per a l'execució local.